**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**
**KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG**



# BÁO CÁO ĐỒ ÁN CUỐI KỲ

**MÔN HỌC: NT531**

**ĐỀ TÀI**

# PERFORMANCE EVALUATION OF AWS VPC PEERING AND TRANSIT GATEWAY

**Đánh giá hiệu năng AWS VPC Peering và Transit Gateway**

**Môn học:** NT531 – Đánh giá hiệu năng mạng máy tính

**Giảng viên hướng dẫn:** Đặng Lê Bảo Chương

| | |
|---|---|
| 1. Nguyễn Ngọc Minh | 24521068 |
| 2. Nguyễn Minh Nhật | 24521253 |
| 3. Nguyễn Trung Kiên | 24520885 |

**TP. Hồ Chí Minh, tháng 10 năm 2026**

# TÓM TẮT

Đồ án đánh giá VPC Peering và AWS Transit Gateway (TGW) trên cùng bốn endpoint EC2 nhằm tách ảnh hưởng của đường kết nối khỏi khác biệt cấu hình máy. Hạ tầng gồm bốn VPC trong `us-east-1`, mỗi VPC có một EC2 `c6i.large` tại cùng Availability Zone. Terraform giữ nguyên endpoint và chuyển 12 route liên VPC giữa full-mesh Peering và TGW hub-and-spoke. Ba bài đo chính gồm RTT khi mạng rảnh, TCP một luồng từ A tới B và TCP fan-in từ A, C, D tới B. Mỗi bộ dữ liệu có năm lượt; raw data, metadata, CPU và bộ đếm ENA được lưu để kiểm chứng lại bằng script.

Trong đợt xác nhận mới, RTT qua TGW cao hơn Peering trong cả 5/5 block cùng cohort; chênh lệch trung bình là 0,3119 ms và toàn bộ 3.000 probe đều phản hồi. Baseline receiver goodput TCP một luồng đạt lần lượt 4,7828 và 4,6083 Gbps. Tổng fan-in xấp xỉ đạt 11,6110 và 11,5905 Gbps, nhưng cả hai phiên đều ghi nhận bộ đếm network allowance tại máy nhận tăng. Kết quả fan-in vì vậy phản ánh đồng thời đường mạng và giới hạn endpoint, không chứng minh hai dịch vụ có năng lực tối đa tương đương. Báo cáo kết luận Peering có lợi thế về latency và single-flow trong cấu hình đã đo, còn TGW có lợi thế về topology, định tuyến tập trung và khả năng mở rộng quản trị. Phạm vi kết luận bị giới hạn bởi số lượt nhỏ, phép đo hai mode nối tiếp, đặc tính burst của EC2 và cửa sổ fan-in chưa đồng bộ tuyệt đối.

Để nối kết metric mạng với chất lượng dịch vụ, nhóm bổ sung workload HTTP mô phỏng hai worker được gọi song song. Ở 800 request/s, 8 slots chỉ đạt 3,0444% đúng hạn qua Peering và 3,2722% qua TGW; 16 slots đạt 100% trong ba block quan sát của mỗi điều kiện. P95 hoàn thành của 8 slots tăng lên khoảng 1,226–1,499 s, trong khi 16 slots giữ quanh 12,7 ms. Log request còn cho phép kiểm tra định luật Little; Grafana/Tempo được dùng để minh họa trace smoke đã lưu, không gắn nhãn trace đó là benchmark 800 request/s. Kết quả cho thấy lựa chọn đường mạng không thể tách khỏi khả năng phục vụ và deadline của ứng dụng.

# LỜI CẢM ƠN

Nhóm xin cảm ơn giảng viên Đặng Lê Bảo Chương đã hướng dẫn các khái niệm về đánh giá hiệu năng, quá trình đo và cách diễn giải giới hạn của số liệu. Nhóm cũng cảm ơn các tài liệu AWS, ESnet iperf3 và giáo trình NT531 đã cung cấp cơ sở để thiết kế thí nghiệm có thể tái tạo. Mọi kết quả trong báo cáo được đối chiếu với raw artifact, metadata và script trong repository trước khi tổng hợp.

# MỤC LỤC

**TÓM TẮT** **1**

**LỜI CẢM ƠN** **i**

**DANH MỤC HÌNH ẢNH** **v**

**DANH MỤC BẢNG BIỂU** **vi**

**DANH MỤC MÃ NGUỒN** **vii**

**DANH MỤC TỪ VIẾT TẮT** **viii**

**CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI** **1**

1.1. Bối cảnh . . . . . 1
1.2. Vấn đề nghiên cứu . . . . . 1
1.3. Mục tiêu . . . . . 1
1.4. Câu hỏi nghiên cứu . . . . . 2
1.5. Phạm vi . . . . . 2
1.6. Đóng góp của đồ án . . . . . 2
1.7. Cấu trúc báo cáo . . . . . 3

**CHƯƠNG 2. NỘI DUNG THỰC HIỆN** **4**

2.1. Amazon Virtual Private Cloud . . . . . 4
2.2. VPC Peering . . . . . 4
2.3. AWS Transit Gateway . . . . . 4
2.4. So sánh kiến trúc Peering và TGW . . . . . 5
2.5. EC2 networking và ENA . . . . . 5
2.6. TCP và các metric . . . . . 5
2.7. Mô hình fan-in . . . . . 6
2.8. Metric thống kê . . . . . 6
2.9. Phân vị và chỉ số đúng hạn . . . . . 6
2.10. Công suất phục vụ danh nghĩa và giới hạn đồng thời . . . . . 7
2.11. Định luật Little cho hàng đợi quan sát . . . . . 7
2.12. Quan sát phân tán . . . . . 7

**CHƯƠNG 3. TRIỂN KHAI VÀ KIỂM THỬ** **8**

3.1. Kiến trúc hệ thống thử nghiệm . . . . . 8
3.2. Kiến trúc VPC Peering . . . . . 8
3.3. Kiến trúc Transit Gateway . . . . . 9

3.4. Điều khiển đường đi bằng Terraform . . . 9
3.5. Đường quản trị và đường đo . . . 10
3.6. Điều kiện benchmark mạng . . . 10
3.7. Công cụ đo và kiểm chứng . . . 10
3.8. Workload ứng dụng và thiết kế slots . . . 11
3.9. Quan sát CPU và ENA . . . 12
3.10. Experiment 1: RTT idle . . . 12
3.11. Experiment 2: TCP single-flow . . . 12
3.12. Experiment 3: TCP fan-in . . . 12
3.13. Quy trình chuyển Peering và TGW . . . 12
3.14. Experiment 4: HTTP deadline và hàng đợi . . . 13
3.15. OpenTelemetry và Grafana . . . 13
3.16. Kết quả chính từ đợt đo xác nhận mới . . . 13
3.16.1. RTT cùng cohort: 5 block và 3.000 probe . . . 13
3.16.2. Deadline ứng dụng: 252.000 request . . . 14
3.16.3. Cơ chế: tail latency đến từ hàng đợi . . . 15
3.16.4. OpenTelemetry và Grafana . . . 16
3.17. Kết luận trực tiếp từ dữ liệu mới . . . 16
3.18. TCP baseline bổ trợ . . . 17
3.19. Khả năng tái tạo . . . 17
3.20. Nguy cơ ảnh hưởng tính hợp lệ . . . 17

**CHƯƠNG 4. CÁC NGHIÊN CỨU LIÊN QUAN 18**
4.1. Hiệu năng mạng trong public cloud . . . 18
4.2. Tài liệu AWS về kết nối liên VPC . . . 18
4.3. Tài liệu về EC2, ENA và công cụ benchmark . . . 18
4.4. Vị trí của đồ án . . . 18
4.5. Độ trễ đuôi và hàng đợi ứng dụng . . . 18
4.6. Observability và giới hạn của công cụ . . . 19

**CHƯƠNG 5. ĐÁNH GIÁ VÀ KẾT LUẬN 20**
5.1. Trả lời câu hỏi nghiên cứu . . . 20
5.1.1. RQ1: RTT khác nhau như thế nào? . . . 20
5.1.2. RQ2: TCP single-flow khác nhau như thế nào? . . . 20
5.1.3. RQ3: Tổng fan-in thay đổi như thế nào? . . . 20
5.1.4. RQ4: Endpoint ảnh hưởng cách diễn giải ra sao? . . . 20
5.1.5. RQ5: Trade-off performance và scalability là gì? . . . 20
5.1.6. RQ6: Network hay worker chi phối deadline? . . . 21
5.2. Đánh giá trade-off kiến trúc . . . 21
5.3. Hạn chế . . . 21

5.4. Hướng phát triển . . . . . 22
5.5. Kết luận . . . . . 22

**DANH MỤC TÀI LIỆU THAM KHẢO** **23**

**PHỤ LỤC A. BẢNG PHÂN CÔNG CÔNG VIỆC** **25**

**PHỤ LỤC B. MINH BẠCH TRONG VIỆC SỬ DỤNG AI** **26**

# DANH MỤC HÌNH ẢNH

3.1 Kiến trúc thử nghiệm sử dụng VPC Peering . . . . . 8
3.2 Kiến trúc thử nghiệm sử dụng AWS Transit Gateway . . . . . 9
3.3 Phân biệt Elastic IP quản trị và private IP dùng cho benchmark . . . . . 10
3.4 Kết quả RTT mới: trung bình block, ECDF của 3.000 probe và hiệu ứng ghép cặp . . . . . 14
3.5 Tỷ lệ đúng hạn và mức lặp lại của hiệu ứng slots trong ba block mới . . . . 15
3.6 P95 end-to-end và phân rã P95 chờ/phục vụ tại worker . . . . . 15
3.7 Pipeline OpenTelemetry Collector–Tempo–Grafana dùng để giải thích request path . . . . . 16

# DANH MỤC BẢNG BIỂU

2.1 Đặc tính kiến trúc, chưa phải kết quả hiệu năng . . . . . 5
3.1 Endpoint được giữ nguyên giữa hai mode . . . . . 8
3.2 Điều kiện của sáu bộ dữ liệu chính . . . . . 10
3.3 Thiết kế workload ứng dụng . . . . . 11
3.4 RTT trung bình trong năm block xác nhận . . . . . 14
3.5 Tóm tắt TCP baseline lịch sử . . . . . 17
A.1 Phân công công việc của nhóm . . . . . 25

# DANH MỤC MÃ NGUỒN

3.1 Lệnh TCP single-flow cốt lõi . . . . . 12

# DANH MỤC TỪ VIẾT TẮT

| Chữ viết tắt | Chữ viết đầy đủ |
|---|---|
| AWS | Amazon Web Services |
| CIDR | Classless Inter-Domain Routing |
| EC2 | Elastic Compute Cloud |
| EIP | Elastic IP address |
| ENA | Elastic Network Adapter |
| IaC | Infrastructure as Code |
| IGW | Internet Gateway |
| MTU | Maximum Transmission Unit |
| PPS | Packets Per Second |
| RTT | Round-Trip Time |
| SG | Security Group |
| TCP | Transmission Control Protocol |
| TGW | AWS Transit Gateway |
| VPC | Virtual Private Cloud |

# CHƯƠNG 1.
# TỔNG QUAN ĐỀ TÀI

## 1.1 Bối cảnh

Kiến trúc cloud thường tách workload thành nhiều VPC để phân chia miền lỗi, quyền quản trị và chính sách bảo mật. Khi các workload cần trao đổi bằng địa chỉ riêng, hệ thống phải bổ sung kết nối liên VPC. Hai phương án AWS được khảo sát trong đồ án là VPC Peering và Transit Gateway (TGW). Peering tạo kết nối trực tiếp giữa từng cặp VPC, trong khi TGW cung cấp một hub định tuyến tập trung cho nhiều attachment [1, 2].

Luồng vấn đề của nghiên cứu được tóm tắt như sau:

Nhiều VPC $\rightarrow$ cần kết nối liên VPC $\rightarrow$ Peering hoặc TGW $\rightarrow$ kiến trúc đường đi khác nhau $\rightarrow$ có thể tạo khác biệt hiệu năng.

Khác biệt kiến trúc không tự động chứng minh khác biệt hiệu năng. Phép so sánh cần giữ nguyên endpoint, vị trí triển khai, tải và công cụ đo; chỉ thay đường định tuyến giữa hai mode.

## 1.2 Vấn đề nghiên cứu

Vấn đề trung tâm là: với cùng endpoint EC2 và cùng điều kiện benchmark, hiệu năng mạng quan sát được giữa VPC Peering và TGW khác nhau như thế nào? Nghiên cứu không đo hai môi trường độc lập vì khác biệt instance có thể làm nhiễu kết quả. Thay vào đó, Terraform chuyển route của cùng bốn EC2 giữa hai đường kết nối rồi kiểm tra lại trước khi đo.

## 1.3 Mục tiêu

Đồ án đánh giá các đại lượng sau:

- RTT khi mạng rảnh giữa A và B;
- receiver goodput của TCP một luồng;
- số TCP retransmission do iperf3 ghi nhận;
- tổng receiver goodput xấp xỉ trong kịch bản fan-in;
- CPU busy tại endpoint gửi và nhận;
- thay đổi các network allowance counter do ENA cung cấp.
- P95 latency và tỷ lệ request hoàn thành trong deadline 50 ms;

– ảnh hưởng của giới hạn đồng thời 8/16 slots tại worker;

– số request chờ trung bình theo định luật Little và trace phân tán.

Mục tiêu phân tích không chỉ là xếp hạng hai giá trị throughput. Báo cáo còn phải xác định khi nào endpoint trở thành bottleneck và ranh giới nào khiến dữ liệu chưa đủ để suy ra năng lực tối đa của dịch vụ AWS.

## 1.4 Câu hỏi nghiên cứu

1. RTT qua Peering và TGW khác nhau bao nhiêu trong cấu hình thử nghiệm?
2. Receiver goodput TCP một luồng khác nhau như thế nào?
3. Khi A, C và D cùng gửi tới B, tổng goodput thay đổi như thế nào?
4. Endpoint EC2/ENA ảnh hưởng ra sao tới cách diễn giải kết quả fan-in?
5. Trade-off giữa performance và khả năng mở rộng kiến trúc là gì?
6. Khi tải ứng dụng tiến gần công suất phục vụ, đường mạng hay giới hạn worker chi phối khả năng đáp ứng deadline?

## 1.5 Phạm vi

Hệ thống thực nghiệm đặt tại region `us-east-1`, Availability Zone `us-east-1a`; gồm bốn VPC và bốn EC2 `c6i.large`. Traffic đo đi qua private IP, dùng TCP và MTU 1500 byte cho baseline. RTT dùng ICMP. Phần lịch sử có năm lượt cho mỗi bài; đợt xác nhận mới có năm block RTT và ba block ứng dụng. Thiết kế 16 block ban đầu được rút ngắn vì ngân sách AWS và không được trình bày như đã hoàn thành.

Phạm vi này không đại diện cho mọi region, Availability Zone, loại instance, MTU, số luồng hay quy mô VPC. Kết quả không phải benchmark năng lực tối đa của Peering hoặc TGW. Chi phí được thảo luận theo mô hình tính phí, không dùng như số liệu hóa đơn của phiên đo.

## 1.6 Đóng góp của đồ án

– Hạ tầng bốn VPC có thể tái tạo bằng Terraform.

– Cùng endpoint được giữ nguyên khi thay đổi đường Peering và TGW.

– Ba kịch bản chính bao phủ RTT, TCP single-flow và TCP fan-in.

– Raw data, metadata, route snapshot, CPU và ENA counter được lưu cùng kết quả.

- Script độc lập tính lại số liệu và kiểm tra tính nhất quán của nguồn.
- Phân tích tách kết quả thực nghiệm khỏi đặc tính kiến trúc và nêu rõ ảnh hưởng của bottleneck tại máy nhận.
- Workload HTTP có kiểm soát cho phép thay 8/16 slots và đo trực tiếp P95, tỷ lệ đúng hạn, queue time và service time.
- Kiểm tra nhất quán hàng đợi bằng $L_q = \lambda_C \overline{W}_q$ trên cùng cohort request ID; dùng OpenTelemetry để minh họa đường đi request.

## 1.7 Cấu trúc báo cáo

Chương 2 trình bày nội dung kỹ thuật trực tiếp phục vụ phép đo. Chương 3 mô tả hạ tầng, thiết kế đối chứng, quy trình benchmark và kết quả thực nghiệm chi tiết. Chương 4 tổng hợp các nghiên cứu và tài liệu liên quan. Chương 5 đánh giá các finding, trả lời câu hỏi nghiên cứu, nêu hạn chế, hướng phát triển và kết luận.

# CHƯƠNG 2.
# NỘI DUNG THỰC HIỆN

## 2.1 Amazon Virtual Private Cloud

Amazon VPC là miền mạng logic cô lập trong AWS. CIDR xác định dải địa chỉ; subnet chia dải đó theo Availability Zone; route table quyết định target cho từng prefix; Security Group (SG) lọc traffic stateful tại network interface [3]. Trong thí nghiệm, bốn CIDR không chồng lấn là điều kiện để route liên VPC có đích xác định.

## 2.2 VPC Peering

VPC Peering nối hai VPC để traffic private đi trực tiếp giữa chúng. Kết nối không có tính bắc cầu: A peering với B và B peering với C không làm A tự động truy cập C qua B [4]. Vì vậy, full mesh của $n$ VPC cần:

$$N_{\text{peering}} = \frac{n(n-1)}{2}. \tag{2.1}$$

Với $n = 4$, hệ thống cần sáu connection: AB, AC, AD, BC, BD và CD. Mỗi route table VPC vẫn cần ba route có hướng tới ba CIDR còn lại, tổng cộng 12 route liên VPC trong cấu hình này.

## 2.3 AWS Transit Gateway

TGW cung cấp mô hình hub-and-spoke. Mỗi VPC tạo một attachment tới hub; TGW route table điều khiển việc chuyển tiếp giữa các attachment [2, 5]. Với bốn VPC, đồ án dùng một TGW và bốn VPC attachment. Mô hình này giảm số quan hệ kết nối logic khi số VPC tăng, nhưng không làm mất yêu cầu cấu hình route đi và route về.

## 2.4 So sánh kiến trúc Peering và TGW

Bảng 2.1. Đặc tính kiến trúc, chưa phải kết quả hiệu năng

| Tiêu chí | VPC Peering | Transit Gateway |
|---|---|---|
| Topology | Point-to-point; full mesh khi mọi VPC cần liên lạc | Hub-and-spoke |
| Tính bắc cầu | Không hỗ trợ | Chuyển tiếp qua hub theo TGW route table |
| Quản trị | Phân tán theo từng cặp và route table VPC | Tập trung attachment và TGW route table |
| Mở rộng kết nối | Số connection tăng theo $n(n-1)/2$ | Số attachment tăng tuyến tính theo $n$; quota áp dụng [6] |
| Chi phí cơ bản | Không tính phí theo giờ cho connection; data transfer áp dụng theo tài liệu AWS [7] | Có phí attachment theo giờ và data processing [8] |

Bảng 2.1 chỉ mô tả thiết kế. RTT và goodput phải được xác định từ thực nghiệm, không suy ra trực tiếp từ số connection.

## 2.5 EC2 networking và ENA

EC2 network performance phụ thuộc loại instance, kích thước, loại traffic và chính sách allowance. Tài liệu AWS phân biệt baseline bandwidth với khả năng burst ở một số instance; mức công bố không đồng nghĩa một TCP flow luôn đạt mức đó [9]. ENA xuất các counter như `bw_in_allowance_exceeded` và `pps_allowance_exceeded`. Counter tăng cho biết traffic đã gặp giới hạn tương ứng tại interface; không được cộng các counter để suy ra packet loss [10].

Trong fan-in, ba sender hội tụ vào B. Ngay cả khi đường kết nối còn dư năng lực, giới hạn bandwidth, PPS, CPU hoặc xử lý TCP tại B vẫn có thể chi phối tổng goodput. Vì vậy CPU và ENA là bằng chứng hỗ trợ xác định bottleneck endpoint, không phải metric phụ.

## 2.6 TCP và các metric

RTT là thời gian khứ hồi của một request và response. Throughput là tốc độ dữ liệu truyền qua hệ thống; receiver goodput trong báo cáo là số bit payload mà phía nhận ghi nhận chia cho thời lượng nhận. Với số byte $B$ và thời gian $T$, giá trị dùng trong báo cáo là:

$$G = \frac{8B}{T \times 10^9} \quad \text{(Gbps)}. \tag{2.2}$$

TCP retransmission là số segment TCP truyền lại do iperf3 báo cáo. Đại lượng này không đồng nhất với phần trăm packet loss. Một single flow còn chịu ảnh hưởng của congestion window, RTT, socket processing và giới hạn endpoint; nhiều concurrent flow có thể phân chia tài nguyên khác với một flow.

## 2.7 Mô hình fan-in

Kịch bản fan-in có ba source cùng gửi tới một receiver:

```
A -----5201---\
C -----5202----> B
D -----5203---/
```

Ba listener tách cổng để tránh tranh chấp một tiến trình server và giữ từng JSON độc lập. Tổng fan-in hiện tại là tổng ba goodput tính trên cửa sổ riêng của từng source. Do wrapper khởi chạy lệch dưới một giây, đây là tổng xấp xỉ, không phải phép tích phân byte trên một cửa sổ giao nhau tuyệt đối.

## 2.8 Metric thống kê

Với $m = 5$ lượt và giá trị $x_i$, mean và sample standard deviation là:

$$\bar{x} = \frac{1}{m}\sum_{i=1}^{m} x_i, \tag{2.3}$$

$$s = \sqrt{\frac{\sum_{i=1}^{m}(x_i - \bar{x})^2}{m-1}}. \tag{2.4}$$

Coefficient of variation là $CV = 100s/\bar{x}$. Chênh lệch tương đối dùng Peering làm mốc được ghi rõ tại từng bảng. Mức chia đều ba flow trong một lượt dùng Jain fairness index [11]:

$$J = \frac{(\sum_{i=1}^{3} g_i)^2}{3\sum_{i=1}^{3} g_i^2}. \tag{2.5}$$

$J$ gần 1 chỉ mô tả mức chia đều trong các cửa sổ đã ghi; không chứng minh max-min fairness hoặc năng lực tối đa của dịch vụ.

## 2.9 Phân vị và chỉ số đúng hạn

Mean có thể che giấu một nhóm request rất chậm. Báo cáo vì vậy dùng phân vị P95 theo quy tắc nearest rank. Với $n$ độ trễ đã sắp tăng dần,

$$k = \lceil 0{,}95n \rceil, \qquad P95 = x_{(k)}. \tag{2.6}$$

P95 trong nghiên cứu ứng dụng chỉ tính trên request hoàn thành thành công. Để không loại request lỗi hoặc trễ khỏi mẫu số chất lượng dịch vụ, nhóm định nghĩa SLI đúng hạn:

$$\widehat{p} = \frac{N_{\text{good}}}{N_{\text{scheduled}}}, \qquad SLI_{\%} = 100\widehat{p}. \tag{2.7}$$

$N_{\text{good}}$ là request thành công và kết thúc không quá 50 ms kể từ lịch gửi; $N_{\text{scheduled}}$ gồm toàn bộ request theo lịch. Ngưỡng 50 ms là lựa chọn của thí nghiệm, không phải SLA của AWS. P50/P95/P99 và SLI/SLO được liên hệ với bài giảng NT531 về Internet Performance và QoS [12].

## 2.10 Công suất phục vụ danh nghĩa và giới hạn đồng thời

Mỗi worker có $M$ slots. Một lời gọi chiếm một slot trong thời gian cấu hình $s = 0{,}010$ s. Với tải theo lịch $\lambda_{\text{cfg}}$, công suất và mức tải danh nghĩa được dùng để chọn điều kiện thử:

$$\mu_{\text{nom}} = \frac{M}{s}, \qquad \rho_{\text{nom}} = \frac{\lambda_{\text{cfg}} s}{M}. \tag{2.8}$$

Với 8 slots, $\mu_{\text{nom}} = 800$ lời gọi/s; 800 request/s làm $\rho_{\text{nom}} = 1$. Với 16 slots, công suất danh nghĩa là 1.600 lời gọi/s và $\rho_{\text{nom}} = 0{,}5$. Đây là mô hình chọn tải, không phải giới hạn AWS, không phải phần trăm CPU và không giả định quá trình đến Poisson.

## 2.11 Định luật Little cho hàng đợi quan sát

Bài giảng Arrival Processes and Queuing Systems trình bày quan hệ $L = \lambda W$ và ví dụ thread pool [13]. Nhóm áp dụng cho phần request đang chờ:

$$\overline{L}_{q,C} = \frac{1}{T_C}\sum_{i=1}^{N} W_{q,i} = \underbrace{\frac{N}{T_C}}_{\lambda_C}\underbrace{\frac{1}{N}\sum_{i=1}^{N} W_{q,i}}_{\overline{W}_q}. \tag{2.9}$$

$T_C$ bao phủ từ lần vào đầu tiên đến lần thoát cuối cùng của cùng cohort, kể cả đuôi xử lý. Vì vậy $\lambda_C = N/T_C$ có thể khác rate cấu hình. Hai vế được tính từ cùng log để kiểm tra nhất quán, không phải hai mẫu thống kê độc lập.

## 2.12 Quan sát phân tán

OpenTelemetry mô hình hóa một request bằng trace và các span; Collector nhận telemetry, Tempo lưu trace, còn Grafana dùng để truy vấn và hiển thị [14]. Pipeline này giúp nhìn vị trí queue/service trong đường gọi A $\rightarrow$ B $\rightarrow$ C,D. Trace smoke được dùng làm minh họa cơ chế; benchmark chính tắt full tracing để tránh thay đổi workload cần so sánh.

# CHƯƠNG 3.
# TRIỂN KHAI VÀ KIỂM THỬ

## 3.1 Kiến trúc hệ thống thử nghiệm

Hạ tầng gồm bốn VPC trong `us-east-1a`. Mỗi VPC có một subnet và một EC2. Các private IP được gán cố định trong Terraform, trình bày tại Bảng 3.1.

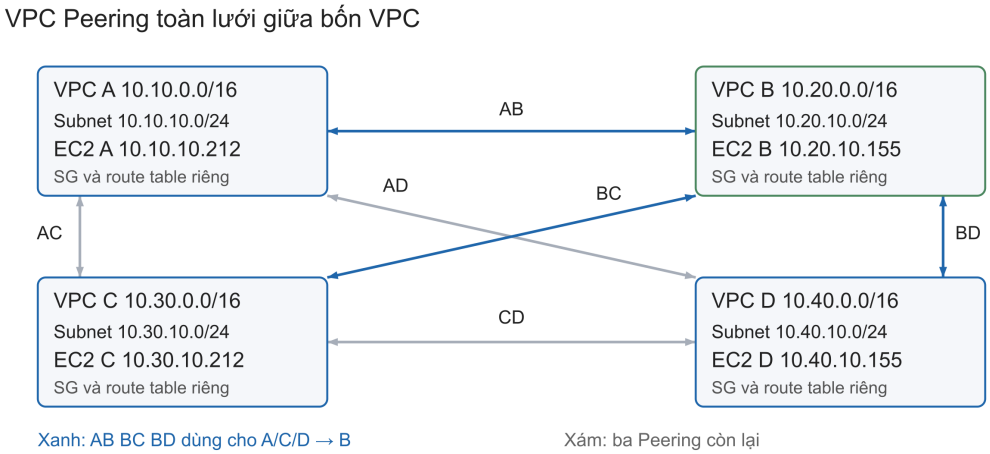
Bảng 3.1. Endpoint được giữ nguyên giữa hai mode

| Nút | VPC CIDR | Subnet CIDR | Private IP |
|---|---|---|---|
| A | 10.10.0.0/16 | 10.10.10.0/24 | 10.10.10.212 |
| B | 10.20.0.0/16 | 10.20.10.0/24 | 10.20.10.155 |
| C | 10.30.0.0/16 | 10.30.10.0/24 | 10.30.10.212 |
| D | 10.40.0.0/16 | 10.40.10.0/24 | 10.40.10.155 |

Bốn máy dùng `c6i.large`, cùng AMI `ami-0d27e0fb3bac4d724`, root volume gp3 8 GiB và IMDSv2 bắt buộc. Elastic IP phục vụ quản trị; traffic benchmark dùng private IP nên EIP và Internet Gateway không nằm trên đường đo.

## 3.2 Kiến trúc VPC Peering

Mode Peering tạo full mesh sáu connection. Mỗi route table có ba route đến ba CIDR còn lại; target của mỗi route là peering connection tương ứng.

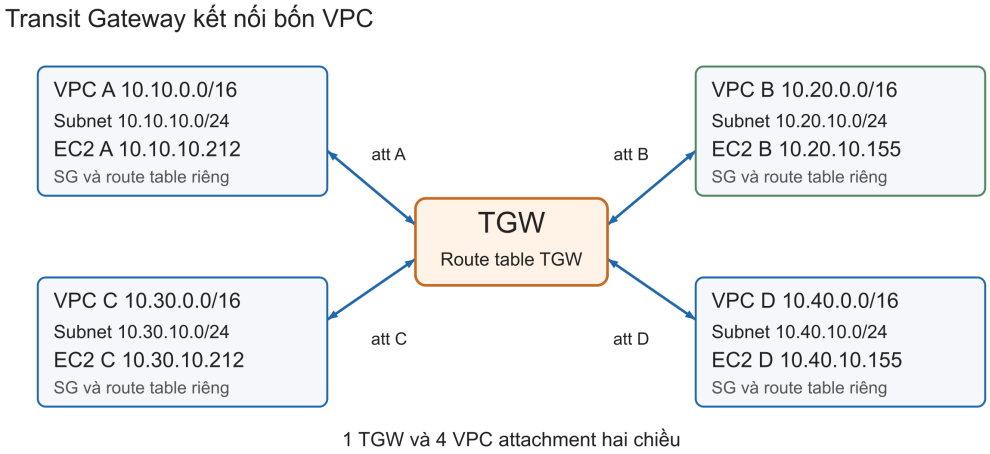



Hình 3.1. Kiến trúc thử nghiệm sử dụng VPC Peering

Hình 3.1 cho thấy endpoint và địa chỉ không thay đổi. Full mesh cung cấp đường trực tiếp cho AB, AC, AD, BC, BD và CD; không dựa vào một VPC trung gian.

## 3.3 Kiến trúc Transit Gateway

Mode TGW giữ nguyên VPC, subnet và EC2 nhưng đổi target của 12 route liên VPC sang một TGW. Mỗi VPC có một attachment tới hub.




Hình 3.2. Kiến trúc thử nghiệm sử dụng AWS Transit Gateway

Hình 3.2 biểu diễn topology hub-and-spoke. Traffic A tới B đi từ route table A tới TGW, qua attachment của B rồi tới EC2 B; chiều về cần route đối ứng.

## 3.4 Điều khiển đường đi bằng Terraform

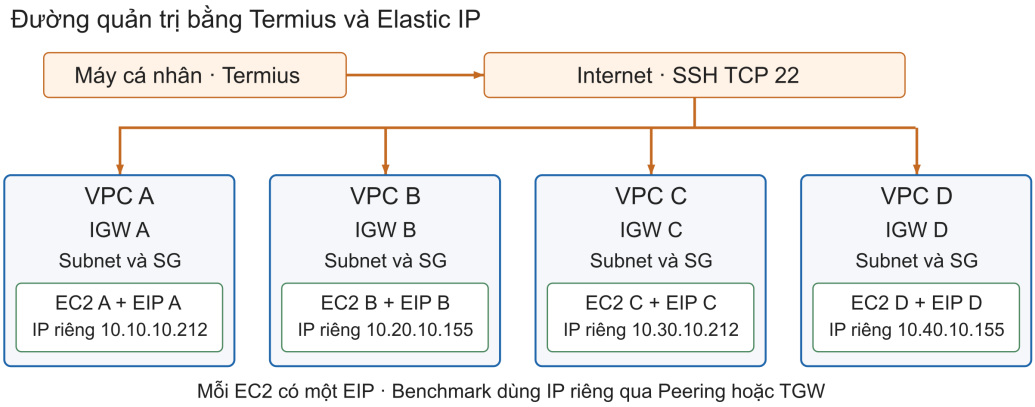
Hai biến quyết định mode:

- `enable_transit_gateway`: tạo TGW và bốn attachment;
- `routing_mode`: nhận `peering` hoặc `tgw` và chọn target cho toàn bộ 12 route liên VPC.

`enable_transit_gateway` + `routing_mode` → 12 route liên VPC → Peering target hoặc TGW target.

Cơ chế này quan trọng cho tính công bằng: phép đo không tạo một tập EC2 mới cho TGW. Terraform `No changes` chỉ chứng minh desired state khớp state đã biết; nó không tự chứng minh packet đang đi đúng đường. Vì vậy quy trình còn lưu route snapshot và chạy connectivity check sau khi đổi mode.

## 3.5 Đường quản trị và đường đo




Hình 3.3. Phân biệt Elastic IP quản trị và private IP dùng cho benchmark

Elastic IP, Internet Gateway và SSH/SSM phục vụ quản trị instance. Traffic benchmark dùng private IP và route liên VPC. Vì vậy EIP không nằm trên đường dữ liệu được so sánh; Hình 3.3 chỉ mô tả control path giúp chuẩn bị và thu thập kết quả.

## 3.6 Điều kiện benchmark mạng

Bảng 3.2. Điều kiện của sáu bộ dữ liệu chính

| **Thuộc tính** | **Giá trị đã lưu** |
|---|---|
| Region/AZ | `us-east-1/us-east-1a` |
| Instance | Bốn EC2 `c6i.large` |
| Đường dữ liệu | Private IPv4 |
| MTU | 1500 byte |
| iperf3 | 3.19.1; TCP cubic |
| TCP single-flow | Một stream, omit 5 s, đo 30 s, cổng 5201 |
| TCP fan-in | Một stream mỗi source, omit 5 s, đo 30 s, cổng 5201–5203 |
| RTT idle | 100 ICMP echo, payload 56 byte, interval 0,2 s |
| Số lượt | Năm lượt cho mỗi bài và mỗi mode |

Metadata ghi NTP đồng bộ trong các phiên fan-in. Hai mode được đo nối tiếp, không randomize hoặc counterbalance đầy đủ; đây là hạn chế phương pháp chứ không phải chi tiết có thể bỏ qua.

## 3.7 Công cụ đo và kiểm chứng

– `ping`: RTT, số request gửi/nhận và loss quan sát.

- `iperf3`: byte nhận, thời lượng, receiver bitrate, retransmission và JSON theo interval [15].
- `mpstat`: lấy %idle của dòng `all`; CPU busy được tính bằng $100 - \%idle$.
- `ethtool -S`: chụp ENA counter trước/sau.
- Terraform: tạo topology và chuyển route có kiểm soát [16].
- Python: kiểm tra checksum, parse raw data, tính metric và đối chiếu giữa JSON, Markdown và figures.

### 3.8 Workload ứng dụng và thiết kế slots

Để kiểm tra liệu network latency có phải yếu tố quyết định chất lượng dịch vụ, nhóm triển khai một workload HTTP nhỏ trên cùng endpoint. A tạo request theo lịch đến gateway B. B gọi C và D song song; mỗi worker thực hiện I/O mô phỏng 10 ms và trả payload JSON 4.096 byte. Hai worker có cùng giới hạn slots, lần lượt 8 và 16. Mỗi điều kiện dùng rate 600 hoặc 800 request/s, deadline 50 ms, timeout 2 s, warm-up 5 s, đo 15 s và drain 15 s. Mỗi block có đủ bốn điều kiện Peering/TGW $\times$ 8/16 slots tại một mức tải; server mới được tạo cho từng điều kiện để tránh carry-over.

Bảng 3.3. Thiết kế workload ứng dụng

| **Thuộc tính** | **Giá trị** | **Ý nghĩa** |
|---|---|---|
| Request rate | 600/800 request/s | 600 là tải dưới danh nghĩa; 800 là tải sát 8 slots |
| Worker slots | 8/16 mỗi worker | Giới hạn đồng thời của ứng dụng, không phải vCPU |
| Service/I/O | 10 ms, 4.096 byte | Workload mô phỏng có kiểm soát |
| Deadline/timeout | 50 ms/2 s | Tách đúng hạn khỏi hoàn thành muộn |
| Chỉ số | P95, SLI, queue/service time | Đo tail latency và cơ chế chờ |

Một request A gọi C và D song song nên 800 request/s tạo khoảng 800 lời gọi/s trên mỗi worker, tức 1.600 lời gọi/s khi cộng hai worker. Bảng này không gán queue ở B cho AWS TGW; B là gateway ứng dụng.

### 3.9 Quan sát CPU và ENA

`mpstat` lấy mẫu CPU trong cửa sổ đo. CPU busy được tính từ dòng `all` theo $100 - \%idle$; đây là mức bận toàn máy, không phải CPU riêng của iperf3. Lệnh `ethtool` với option `-S` trên interface `ens5` chụp trước/sau năm counter:

- `bw_in_allowance_exceeded` và `bw_out_allowance_exceeded`;
- `pps_allowance_exceeded`;
- `conntrack_allowance_exceeded`;
- `linklocal_allowance_exceeded`.

### 3.10 Experiment 1: RTT idle

A gửi 100 ICMP echo tới B qua private IP. Mỗi lượt lưu raw output và summary gồm min/avg/max/mdev. Metric so sánh là mean của năm giá trị RTT avg. Payload 56 byte và interval 0,2 s được giữ giống nhau giữa hai mode.

### 3.11 Experiment 2: TCP single-flow

A chạy một TCP stream tới listener B:5201. Command cốt lõi trong script là:

Mã nguồn 3.1: Lệnh TCP single-flow cốt lõi

```
iperf3 -c 10.20.10.155 -p 5201 -P 1 -O 5 -t 30 -i 1 -J \
  --get-server-output > "$run_file"
```

Goodput lấy từ phía nhận, không lấy bitrate phía gửi. CPU và ENA được thu ở cả endpoint theo phạm vi mà artifact cho phép.

### 3.12 Experiment 3: TCP fan-in

A, C và D lần lượt gửi tới B trên cổng 5201, 5202 và 5203. Ba listener độc lập giúp server nhận đồng thời và từng source có JSON riêng. Mỗi lượt được lên lịch cách nhau 45 giây; thời gian bắt đầu wrapper giữa source lệch khoảng 0,78–0,82 giây trong phiên TGW. Vì cửa sổ nội bộ không trùng tuyệt đối, tổng ba mean được gọi là tổng xấp xỉ.

### 3.13 Quy trình chuyển Peering và TGW

1. Áp dụng mode định tuyến bằng Terraform.
2. Kiểm tra 12 route liên VPC và target hai chiều.

3. Chạy connectivity check giữa endpoint và các cổng đo.
4. Chuẩn bị EC2: MTU, NTP, listener, công cụ và tiến trình cũ.
5. Chạy benchmark với tham số cố định.
6. Thu raw output, CPU, ENA, timestamps và metadata.
7. Sao lưu artifact và manifest/checksum.
8. Chạy script verify và build báo cáo kết quả.

## 3.14 Experiment 4: HTTP deadline và hàng đợi

Loader A gửi request duy nhất có request ID. Gateway B truyền context ID tới C và D, chờ cả hai worker rồi trả response. Mỗi request lưu timestamp scheduled, entry, service start, service end và exit. Từ các timestamp này, script tính đúng hạn, P95 hoàn thành, thời gian chờ và thời gian phục vụ. Hàng đợi được kiểm tra lại bằng sample-path Little trên cùng cohort.

## 3.15 OpenTelemetry và Grafana

Collector nhận OTLP từ workload demo và chuyển trace sang Tempo; Grafana hiển thị bảng study và waterfall. Trace được gắn nhãn smoke/historical, gồm 12 trace và 120 span đã lưu. Pipeline chứng minh khả năng quan sát queue/service, nhưng không được dùng để thay thế kết quả 800 request/s vì full tracing tắt trong primary benchmark.

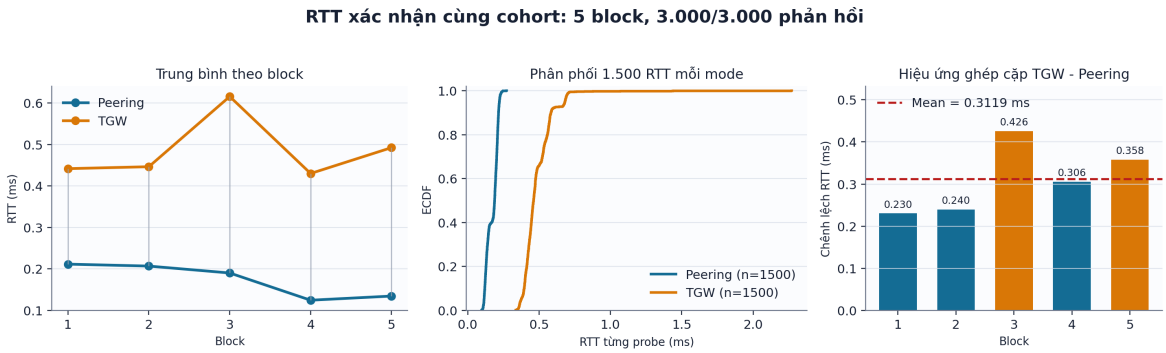
## 3.16 Kết quả chính từ đợt đo xác nhận mới

Phần này là trọng tâm thực nghiệm của báo cáo. Khác với các phiên baseline ban đầu, đợt mới giữ nguyên cohort EC2, lưu route snapshot, đảo thứ tự mode giữa các block và phân tích trực tiếp raw artifact. Dữ liệu gồm năm block RTT và ba block ứng dụng, không trộn với dữ liệu pilot.

### 3.16.1 RTT cùng cohort: 5 block và 3.000 probe

Mỗi block đo ba lượt cho từng mode, mỗi lượt 100 ICMP echo. Toàn bộ 3.000/3.000 probe nhận được phản hồi. Hiệu ứng ghép cặp của block $j$ là

$$\Delta_j = \overline{RTT}_{\mathrm{TGW},j} - \overline{RTT}_{\mathrm{Peering},j}. \tag{3.1}$$




Hình 3.4. Kết quả RTT mới: trung bình block, ECDF của 3.000 probe và hiệu ứng ghép cặp

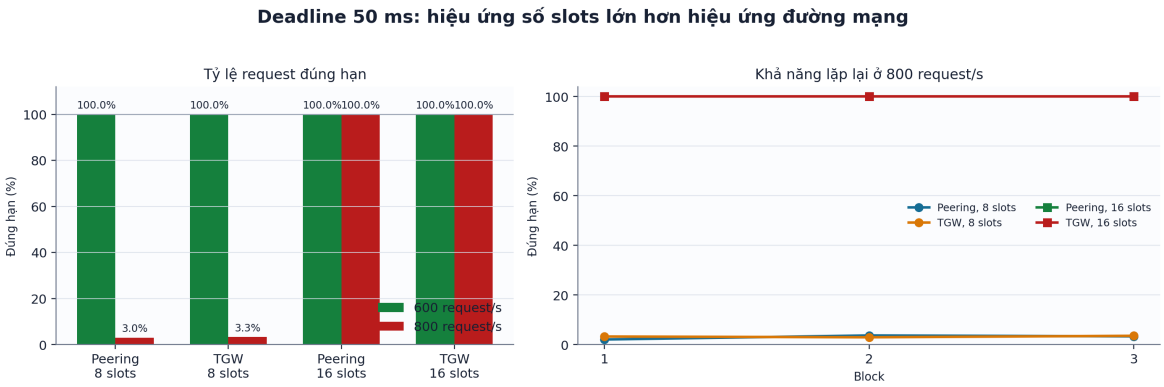
Bảng 3.4. RTT trung bình trong năm block xác nhận

| **Block** | **Peering (ms)** | **TGW (ms)** | $\Delta_j$ **(ms)** |
|---|---|---|---|
| 1 | 0,2113 | 0,4417 | 0,2303 |
| 2 | 0,2067 | 0,4463 | 0,2397 |
| 3 | 0,1900 | 0,6157 | 0,4257 |
| 4 | 0,1243 | 0,4300 | 0,3057 |
| 5 | 0,1343 | 0,4923 | 0,3580 |
| Trung bình | 0,1733 | 0,4852 | 0,3119 |
| Trung vị hiệu ứng | – | – | 0,3057 |

TGW có RTT cao hơn Peering trong cả 5/5 block; hiệu ứng nằm trong khoảng 0,2303–0,4257 ms, trung bình 0,3119 ms. ECDF trong Hình 3.4 dùng đủ 1.500 probe mỗi mode, cho thấy hai phân phối tách rõ trong cohort này chứ không chỉ khác nhau ở một giá trị mean. Kết luận này chỉ áp dụng cho cấu hình đã đo; $\Delta_j$ là chênh lệch end-to-end, không phải thời gian xử lý nội bộ của TGW.

### 3.16.2 Deadline ứng dụng: 252.000 request

Ba block hợp lệ bao phủ Peering/TGW, 8/16 slots và 600/800 request/s. Tổng cộng có 252.000 request theo lịch, tương ứng 504.000 lời gọi worker C/D. Không điều kiện nào ghi nhận HTTP/JSON fail; request hoàn thành sau 50 ms vẫn bị tính là không đúng hạn.

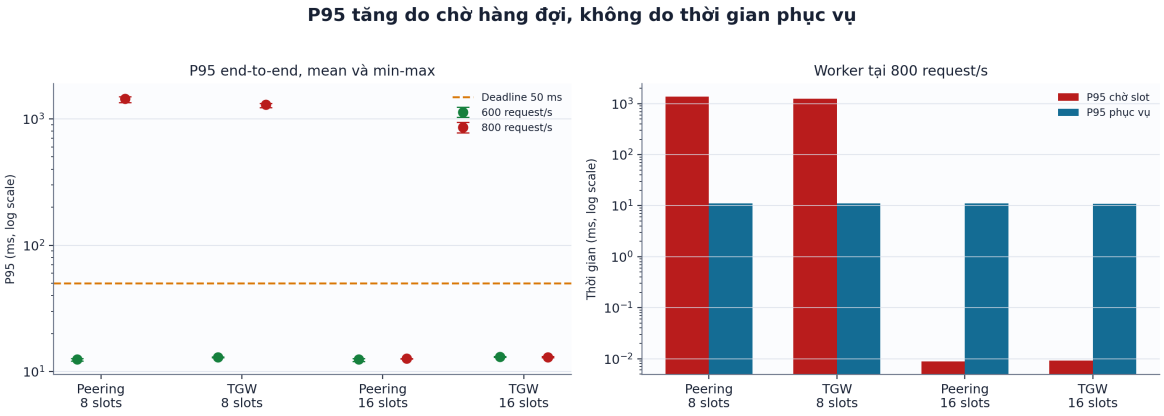



Hình 3.5. Tỷ lệ đúng hạn và mức lặp lại của hiệu ứng slots trong ba block mới

Ở 600 request/s, cả tám cấu hình theo mode và slots đều đạt 100% đúng hạn trong dữ liệu quan sát. Ở 800 request/s, 8 slots chỉ đạt 1.096/36.000 request đúng hạn qua Peering và 1.178/36.000 qua TGW, lần lượt 3,0444% và 3,2722%. Khi tăng lên 16 slots, cả hai mode đạt 36.000/36.000 request đúng hạn.

Hiệu ứng 16 − 8 slots tại 800 request/s, trung bình qua hai route và ba block, là 96,8417 điểm phần trăm; từng block nằm trong khoảng 96,5458–97,3167 điểm phần trăm. Vì Peering và TGW đều thất bại với 8 slots và đều phục hồi với 16 slots, đổi route không giải quyết được giới hạn phục vụ của worker trong workload này.

### 3.16.3 Cơ chế: tail latency đến từ hàng đợi




Hình 3.6. P95 end-to-end và phân rã P95 chờ/phục vụ tại worker

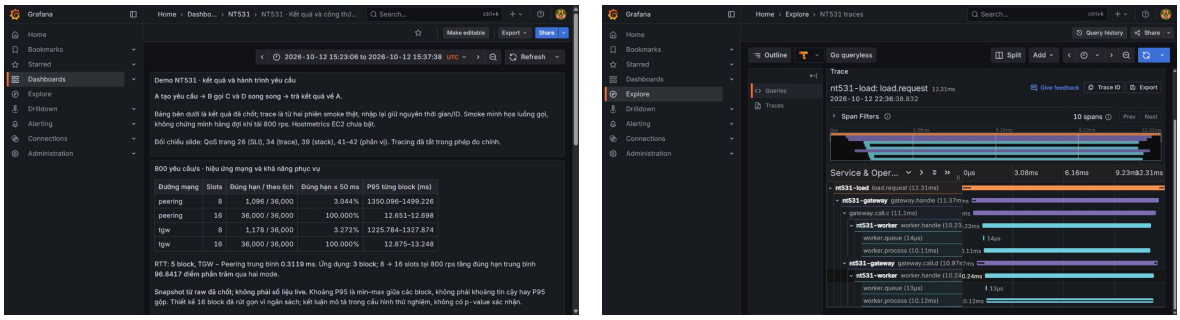
Với 800 request/s và 8 slots, P95 end-to-end nằm trong khoảng 1.225,784–1.499,226 ms, vượt xa deadline 50 ms. P95 chờ slot vào khoảng 1,2–1,4 s, trong khi P95 phục vụ vẫn chỉ khoảng 11 ms. Với 16 slots, P95 chờ giảm xuống dưới 0,013 ms và P95 end-to-end trở lại khoảng 12,651–13,248 ms. Hình 3.6 vì vậy cung cấp chuỗi bằng chứng nhất quán: tỷ lệ đúng hạn tăng, tail latency giảm, thời gian chờ biến mất, còn thời gian phục vụ gần như giữ nguyên.

Ví dụ block 3, worker C, Peering, 800 request/s: với 8 slots, $\lambda_C = 732{,}2866$ lời gọi/s và $\overline{W}_q = 0{,}7009511$ s. Định luật Little trên cohort log hữu hạn cho

$$\overline{L}_q = \lambda_C \overline{W}_q = 732{,}2866 \times 0{,}7009511 \approx 513{,}2971. \tag{3.2}$$

Với 16 slots ở cùng block/mode/worker, $\lambda_C = 799{,}5313$ lời gọi/s, $\overline{W}_q = 0{,}000005239$ s và $\overline{L}_q \approx 0{,}004189$. Hai giá trị cho thấy backlog hàng trăm lời gọi đã chuyển thành hàng đợi gần như trống. Phép tính dùng cùng cửa sổ log và được kiểm tra bằng diện tích occupancy; không giả định arrival Poisson hay thời gian phục vụ exponential.

### 3.16.4 OpenTelemetry và Grafana



(a) Dashboard tổng hợp (b) Waterfall A–B–C/D

Hình 3.7. Pipeline OpenTelemetry Collector–Tempo–Grafana dùng để giải thích request path

Pipeline đã replay và kiểm tra 12 trace thật, tổng 120 span. Waterfall cho thấy B gọi C/D song song và cho phép phân biệt queue với service. Trace này là smoke lịch sử để minh họa cấu trúc request, không phải mẫu benchmark 800 request/s và không được dùng để tính P95 chính.

## 3.17 Kết luận trực tiếp từ dữ liệu mới

Ba kết quả mới có mức bằng chứng khác nhau nhưng cùng trả lời một câu hỏi rõ:

1. Trong 5/5 block cùng cohort, TGW tăng RTT so với Peering; hiệu ứng ghép cặp trung bình là 0,3119 ms.

2. Tại 800 request/s, đổi Peering/TGW không cứu được cấu hình 8 slots; tăng lên 16 slots cải thiện đúng hạn trung bình 96,8417 điểm phần trăm.

3. P95 phục vụ giữ quanh 11 ms trong khi P95 chờ thay đổi hơn năm bậc độ lớn. Hàng đợi ứng dụng là cơ chế chính tạo tail latency trong workload này.

Đây là đóng góp mới của đồ án. Kết quả không tuyên bố Peering hoặc TGW thắng mọi workload; nó chỉ ra khi nào chênh lệch đường mạng đáng chú ý và khi nào nút thắt ứng dụng lớn đến mức lựa chọn route không còn quyết định SLI.

### 3.18 TCP baseline bổ trợ

Các phép đo TCP trước đợt xác nhận được giữ làm baseline mạng. Chúng không được trộn vào năm block RTT hoặc ba block ứng dụng mới vì khác thời điểm và thiết kế.

Bảng 3.5. Tóm tắt TCP baseline lịch sử

| Metric | Peering | TGW | Diễn giải |
|---|---|---|---|
| Single-flow receiver goodput | 4,782804 Gbps | 4,608265 Gbps | Peering cao hơn 3,7875% |
| Fan-in tổng xấp xỉ | 11,611009 Gbps | 11,590520 Gbps | Chênh 0,17646% |
| CPU busy receiver B | 59,1802% | 56,8751% | Không loại trừ bottleneck một core/ENA |
| Jain index fan-in | 0,999133 | 0,984494 | Mô tả chia sẻ trong phiên |

Single-flow cho thấy Peering có receiver goodput cao hơn trong hai phiên đã đo. Tuy nhiên retransmission khác mạnh giữa hai phiên: Peering ghi 48.253–57.185, TGW ghi 0–671. Vì vậy không quy toàn bộ chênh lệch goodput cho network service.

Trong fan-in, tổng goodput của hai mode gần nhau nhưng receiver B đồng thời ghi delta lớn ở `bw_in_allowance_exceeded` và `pps_allowance_exceeded`. Ba sender cùng hội tụ vào B nên endpoint allowance có thể che khác biệt của network path. Kết quả fan-in không phải equivalence test và không chứng minh hai dịch vụ có throughput bằng nhau.

Baseline TCP trả lời khía cạnh throughput; đợt mới trả lời khả năng lặp lại của RTT và cơ chế tail latency ứng dụng. Hai lớp bằng chứng bổ sung cho nhau, không được dùng thay thế lẫn nhau.

### 3.19 Khả năng tái tạo

Terraform mô tả topology; shell script mô tả workload; raw artifact giữ đầu vào; JSON phân tích giữ số đầy đủ và SHA256. Báo cáo này dùng số đã được tính lại từ nguồn thay vì nhập tay từ biểu đồ. Tuy vậy, tái tạo cùng code không bảo đảm số đo giống hệt vì nền tảng cloud có biến thiên theo thời gian và host.

### 3.20 Nguy cơ ảnh hưởng tính hợp lệ

Thiết kế hiện tại có năm lượt cho mỗi bộ chính; Peering và TGW được chạy nối tiếp, chưa counterbalance đầy đủ. Bandwidth burst, placement và nhiễu nền của EC2 không được kiểm soát trực tiếp. Trong fan-in, ba wrapper không bắt đầu tuyệt đối đồng thời và ENA tại B được chụp cho cả phiên. Receiver B có thể trở thành bottleneck dùng chung. Các điểm này giới hạn causal attribution: kết quả là end-to-end observation của cấu hình đã đo, không phải service limit riêng của Peering hoặc TGW.

# CHƯƠNG 4.
# CÁC NGHIÊN CỨU LIÊN QUAN

## 4.1 Hiệu năng mạng trong public cloud

CloudCmp đề xuất benchmark nhiều chiều để so sánh public cloud và chỉ ra rằng kết luận phụ thuộc workload, metric và thời điểm đo [17]. Cách tiếp cận này hỗ trợ lựa chọn phương pháp của đồ án: không dùng một metric duy nhất để kết luận toàn bộ giải pháp kết nối.

Wang và Ng đo ảnh hưởng của virtualization lên network performance của Amazon EC2, cho thấy biến thiên và overhead của nền tảng ảo hóa cần được xem xét khi giải thích latency và throughput [18]. Nghiên cứu đó tạo bối cảnh cho việc lưu CPU, retransmission và nhiều lượt đo, nhưng không trực tiếp so sánh VPC Peering với TGW trên cùng endpoint.

## 4.2 Tài liệu AWS về kết nối liên VPC

AWS mô tả VPC Peering là kết nối point-to-point, không bắc cầu; TGW là router khu vực kết nối VPC và mạng ngoài AWS qua attachment [1, 2]. Tài liệu route, quota và pricing xác định đặc tính quản trị, scale và cost model, nhưng không công bố một giá trị RTT hoặc TCP goodput áp dụng cho mọi workload.

## 4.3 Tài liệu về EC2, ENA và công cụ benchmark

Tài liệu EC2 network bandwidth giải thích giới hạn của instance [9]. Tài liệu ENA mô tả ý nghĩa của allowance counter [10]. iperf3 cung cấp JSON gồm byte, thời gian, bitrate phía nhận và retransmission; công cụ đo end-to-end nên không cô lập riêng một nút mạng [15].

## 4.4 Vị trí của đồ án

Đồ án sử dụng cùng bốn EC2, cùng Availability Zone, private IP, MTU và tham số tải; chỉ chuyển 12 route giữa Peering và TGW. Ngoài RTT và TCP single-flow, nghiên cứu còn quan sát fan-in, CPU và ENA, đồng thời giữ raw artifact để tính lại. Thiết kế này giảm một số confounder so với hai environment độc lập nhưng không loại bỏ biến thiên host, không thay thế benchmark đa region và không tạo cơ sở để tuyên bố đây là nghiên cứu đầu tiên về chủ đề này.

## 4.5 Độ trễ đuôi và hàng đợi ứng dụng

Dean và Barroso chỉ ra rằng latency đuôi trở nên quan trọng khi một request phụ thuộc nhiều thành phần; chậm ở một nhánh có thể kéo dài toàn bộ request [19]. Workload của đồ án chỉ

có hai worker nên không tái tạo quy mô hệ thống trong nghiên cứu đó, nhưng cùng nguyên lý "chờ nhánh chậm" giải thích vì sao cần P95 thay vì chỉ nhìn mean.

Định luật Little liên hệ số đối tượng trung bình trong hệ thống, tốc độ đến và thời gian lưu [20, 21]. Đồ án không áp công thức M/M/1 để dự đoán AWS vì chưa kiểm tra giả định Poisson/exponential. Thay vào đó, nhóm dùng sample path hữu hạn của cùng cohort request ID để kiểm tra $L_q = \lambda_C \overline{W}_q$ trực tiếp từ timestamp.

## 4.6 Observability và giới hạn của công cụ

OpenTelemetry cung cấp API/SDK và Collector trung lập để truyền traces, metrics và logs [14]. Trong đồ án, Collector–Tempo–Grafana giúp minh họa đường gọi và thời gian của span. Công cụ quan sát không thay thiết kế lấy mẫu: replay 12 trace smoke không được dùng như hàng nghìn request độc lập hoặc như bằng chứng của benchmark tải cao.

# CHƯƠNG 5.
# ĐÁNH GIÁ VÀ KẾT LUẬN

## 5.1 Trả lời câu hỏi nghiên cứu

Hai lớp kết quả được tách rõ. Phần lịch sử mô tả RTT/TCP/fan-in; phần xác nhận mới dùng cùng cohort và block cân bằng để đánh giá RTT, sau đó dùng workload HTTP để kiểm tra ảnh hưởng của khả năng phục vụ ứng dụng.

### 5.1.1 RQ1: RTT khác nhau như thế nào?

Trong đợt xác nhận mới, TGW có RTT cao hơn Peering trong cả 5/5 block cùng cohort. Hiệu ứng ghép cặp TGW trừ Peering là 0,2303–0,4257 ms, trung bình 0,3119 ms trên 3.000 probe mỗi mode. Baseline lịch sử cũng cho cùng hướng chênh lệch (0,2078 ms so với 0,5630 ms), nhưng giá trị baseline không được gộp vào thống kê block mới. Kết luận áp dụng cho cấu hình end-to-end đã đo, không phải tuyên bố Peering luôn có latency thấp hơn TGW trong mọi môi trường.

### 5.1.2 RQ2: TCP single-flow khác nhau như thế nào?

Receiver goodput mean đạt 4,782804 Gbps qua Peering và 4,608265 Gbps qua TGW. Theo định nghĩa comparison của independent review, Peering cao hơn 3,7875% trong các phiên đo hiện tại. Retransmission khác biệt lớn giữa hai phiên nhưng chưa đủ bằng chứng để quy nguyên nhân cho riêng network service.

### 5.1.3 RQ3: Tổng fan-in thay đổi như thế nào?

Tổng goodput xấp xỉ đạt 11,611009 Gbps qua Peering và 11,590520 Gbps qua TGW, chênh lệch 0,17646% theo Peering làm mốc. Hai giá trị gần nhau trong dữ liệu hiện có, nhưng chưa có equivalence test và cửa sổ source chưa trùng tuyệt đối.

### 5.1.4 RQ4: Endpoint ảnh hưởng cách diễn giải ra sao?

CPU busy B khoảng 59,18% và 56,88%; đồng thời bandwidth/PPS allowance counter tại B tăng trong cả hai phiên fan-in. Vì ba source dùng chung receiver, tổng goodput có thể bị giới hạn bởi EC2/ENA tại B. Fan-in vì vậy không cô lập riêng khả năng chuyển tiếp của Peering hoặc TGW.

### 5.1.5 RQ5: Trade-off performance và scalability là gì?

Peering có RTT và single-flow goodput tốt hơn trong cấu hình đã đo, đồng thời phù hợp khi số VPC nhỏ và cần kết nối trực tiếp. TGW đổi lại bằng topology hub-and-spoke, định tuyến

tập trung và số attachment tăng tuyến tính. Lựa chọn phụ thuộc latency target, quy mô VPC, governance, cost và khả năng vận hành.

### 5.1.6 RQ6: Network hay worker chi phối deadline?

Ở 600 request/s, mọi điều kiện đạt 100% đúng hạn trong ba block. Ở 800 request/s, cả Peering và TGW đều chỉ đạt khoảng 3% đúng hạn với 8 slots, nhưng đạt 100% trong dữ liệu quan sát với 16 slots. Mean hiệu ứng 16 trừ 8 slots là 96,8417 điểm phần trăm. Queue audit cho thấy 8 slots tạo hàng trăm lời gọi chờ trung bình, trong khi 16 slots gần như không có backlog. Vì vậy, trong workload này, giới hạn đồng thời của worker là yếu tố chi phối deadline tại tải 800 request/s; chênh lệch route vẫn tồn tại nhưng nhỏ hơn hiệu ứng hàng đợi.

## 5.2 Đánh giá trade-off kiến trúc

Peering không bắc cầu. Full mesh bốn VPC cần sáu connection; khi thêm VPC, số quan hệ và route cần quản trị tăng nhanh. Lợi thế là topology trực tiếp và không có phí attachment theo giờ.

TGW thêm một lớp hub, VPC attachment, TGW route table và chi phí xử lý dữ liệu. Đổi lại, network team có điểm kiểm soát tập trung, tách policy tốt hơn và không cần tạo peering cho từng cặp. Đây là architectural advantage, không phải bằng chứng TGW có throughput hoặc latency tốt hơn.

## 5.3 Hạn chế

- Đợt xác nhận có năm block RTT và ba block ứng dụng; số block ứng dụng vẫn nhỏ hơn protocol 16 block ban đầu do ngân sách.
- Peering và TGW được đo nối tiếp, chưa counterbalance/randomize đầy đủ.
- Burst bandwidth, placement và nhiễu nền của EC2 có thể ảnh hưởng.
- Fan-in cộng mean từ các cửa sổ riêng, chưa căn common overlap window.
- ENA delta của B bao phủ cả phiên, chưa gắn cho từng lượt.
- Receiver B có dấu hiệu bandwidth/PPS allowance bottleneck.
- Phạm vi chỉ gồm một Region, một AZ và một loại/kích thước instance.
- Workload chỉ gồm idle RTT, TCP single-flow và TCP fan-in baseline.
- Workload HTTP dùng I/O mô phỏng 10 ms và hai worker; chưa đại diện cho database, cache hoặc microservice sản xuất.

- P95 tính trên request thành công; SLI deadline dùng toàn bộ request theo lịch. Request trong cùng phiên có thể phụ thuộc nhau.
- Trace Grafana là smoke lịch sử, không phải trace benchmark 800 rps; host metrics OpenTelemetry trên EC2 chưa được triển khai.
- Kết quả chưa xác định performance tối đa hoặc equivalence của dịch vụ.

## 5.4 Hướng phát triển

Các hạng mục sau là future work, chưa có kết quả thực nghiệm trong báo cáo:

- nếu ngân sách cho phép, hoàn thành repeated balanced trials nhiều ngày;
- mở rộng số block RTT và đo RTT under load, báo cáo P50/P95 và ECDF;
- phân tích TCP congestion window và RTT theo thời gian;
- tính fan-in trên common overlapping window;
- thử receiver lớn hơn để kiểm tra giả thuyết endpoint bottleneck;
- trace lấy mẫu có kiểm soát ở tải cao và bổ sung host metrics;
- xây dựng cost/performance analysis bằng bảng giá chốt theo ngày.

## 5.5 Kết luận

Trong môi trường bốn EC2 `c6i.large` cùng AZ, MTU 1500 và workload đã đo, Peering cho thấy lợi thế về RTT idle và receiver goodput TCP một luồng. TGW đem lại lợi thế về topology hub-and-spoke, định tuyến tập trung và khả năng mở rộng quan hệ kết nối khi số VPC tăng.

Fan-in đạt tổng xấp xỉ gần nhau, nhưng network allowance counter tại B tăng ở cả hai mode. Kết quả phù hợp với khả năng endpoint chi phối aggregate throughput, không đủ để tuyên bố Peering và TGW có performance giống nhau. Lựa chọn giải pháp cần xét đồng thời latency, scale, governance, cost và workload; không nên dựa riêng vào một giá trị throughput.

Đợt workload ứng dụng bổ sung một kết luận quan trọng: network path có RTT thấp không tự bảo đảm request đúng hạn. Khi 8 slots vận hành sát công suất danh nghĩa, backlog làm P95 tăng lên hơn một giây qua cả hai route; tăng lên 16 slots đưa P95 về khoảng 13 ms và toàn bộ request quan sát trở lại deadline. Vì vậy quyết định kiến trúc phải xét đồng thời đường mạng, khả năng phục vụ, hàng đợi và SLI của ứng dụng. Đây là kết luận có điều kiện trong cấu hình đã đo, không phải tuyên bố Peering hoặc TGW thắng tổng quát.

# DANH MỤC TÀI LIỆU THAM KHẢO


[1] Amazon Web Services. *What is VPC peering?* URL: https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html (visited on 10/11/2026).

[2] Amazon Web Services. *What is AWS Transit Gateway?* URL: https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html (visited on 10/11/2026).

[3] Amazon Web Services. *What is Amazon VPC?* URL: https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html (visited on 10/11/2026).

[4] Amazon Web Services. *VPC peering limitations*. URL: https://docs.aws.amazon.com/vpc/latest/peering/invalid-peering-configurations.html (visited on 10/11/2026).

[5] Amazon Web Services. *Transit gateway route tables*. URL: https://docs.aws.amazon.com/vpc/latest/tgw/tgw-route-tables.html (visited on 10/11/2026).

[6] Amazon Web Services. *Quotas for your transit gateways*. URL: https://docs.aws.amazon.com/vpc/latest/tgw/transit-gateway-quotas.html (visited on 10/11/2026).

[7] Amazon Web Services. *Amazon VPC pricing*. URL: https://aws.amazon.com/vpc/pricing/ (visited on 10/11/2026).

[8] Amazon Web Services. *AWS Transit Gateway pricing*. URL: https://aws.amazon.com/transit-gateway/pricing/ (visited on 10/11/2026).

[9] Amazon Web Services. *Amazon EC2 instance network bandwidth*. URL: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-network-bandwidth.html (visited on 10/11/2026).

[10] Amazon Web Services. *Monitor network performance for your EC2 instance*. URL: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/monitoring-network-performance-ena.html (visited on 10/11/2026).

[11] Raj Jain, Dah-Ming Chiu, and William R. Hawe. *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems*. Tech. rep. DEC-TR-301. Digital Equipment Corporation, 1984. URL: https://www.cs.wustl.edu/~jain/papers/ftp/fairness.pdf (visited on 10/11/2026).

[12] Khoa Mạng máy tính và Truyền thông, UIT. *NT531 - Internet Performance, QoS and Cloud-Native Metrics*. Tài liệu bài giảng do sinh viên cung cấp; tham chiếu các trang 26–27, 31, 39 và 41–42. 2026.

[13] Khoa Mạng máy tính và Truyền thông, UIT. *NT531 - Lecture 02: Arrival Processes and Queuing Systems*. Tài liệu bài giảng do sinh viên cung cấp; tham chiếu các trang 3 và 62–66. 2026.

[14] OpenTelemetry Authors. *OpenTelemetry Documentation*. URL: https://opentelemetry.io/docs/ (visited on 10/13/2026).

[15] ESnet. *Invoking iperf3*. URL: https://software.es.net/iperf/invoking.html (visited on 10/11/2026).

[16] HashiCorp. *Terraform AWS Provider documentation*. URL: https://registry.terraform.io/providers/hashicorp/aws/latest/docs (visited on 10/11/2026).

[17] Ang Li, Xiaowei Yang, Srikanth Kandula, and Ming Zhang. "CloudCmp: Comparing Public Cloud Providers". In: *Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement*. 2010, pp. 1–14. DOI: 10.1145/1879141.1879143.

[18] Guohui Wang and T. S. Eugene Ng. "The Impact of Virtualization on Network Performance of Amazon EC2 Data Center". In: *2010 Proceedings IEEE INFOCOM*. 2010, pp. 1–9. DOI: 10.1109/INFCOM.2010.5461931.

[19] Jeffrey Dean and Luiz André Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794.

[20] John D. C. Little. "Little's Law as Viewed on Its 50th Anniversary". In: *Operations Research* 59.3 (2011), pp. 536–549. DOI: 10.1287/opre.1110.0940.

[21] Karl Sigman. *Notes on Little's Law*. 2009. URL: https://www.columbia.edu/~ks20/stochastic-I/stochastic-I-LL.pdf (visited on 10/13/2026).

# PHỤ LỤC A.
# BẢNG PHÂN CÔNG CÔNG VIỆC

Nhóm gồm ba thành viên. Tỷ lệ dưới đây là phân công thực hiện cho đồ án; nhóm có thể điều chỉnh nếu biên bản phân công chính thức của lớp yêu cầu cách ghi khác.

Bảng A.1. Phân công công việc của nhóm

| STT | Họ và tên | MSSV | Nội dung công việc | Tỷ lệ |
|---|---|---|---|---|
| 1 | Nguyễn Ngọc Minh | 24521068 | Điều phối, Terraform, thiết kế thí nghiệm, phân tích và trình bày | 34% |
| 2 | Nguyễn Minh Nhật | 24521253 | Chuẩn bị workload, thu raw TCP/HTTP, kiểm tra log và biểu đồ | 33% |
| 3 | Nguyễn Trung Kiên | 24520885 | Kiểm chứng route, quan sát Grafana, rà soát báo cáo và demo | 33% |

# PHỤ LỤC B.
# MINH BẠCH TRONG VIỆC SỬ DỤNG AI

Công cụ AI được dùng để hỗ trợ tổ chức cấu trúc, biên tập diễn đạt, giải thích công thức và kiểm tra tính nhất quán của source báo cáo. AI không tạo số đo mới và không thay đổi raw experimental evidence. Các claim về kiến trúc được đối chiếu với Terraform và tài liệu AWS; phương pháp được đối chiếu với measurement scripts và metadata; số liệu được truy ngược về raw artifact cùng các file verification đã lưu.

Nhóm chịu trách nhiệm cuối cùng đối với nội dung nộp, citation, số liệu và kết luận; mỗi thành viên phải tự giải thích được thiết kế, công thức và giới hạn của kết quả khi bảo vệ.